

El Batch y el Epoch (aclaraciones)

Prof. M.A.Valle
E-MAIL: mauricio.valle.b@uai.cl

Tipos de gradient descents

En general hay 3 tipos de GDs que difieren en cuántos datos (instancias) se usan para ir calculando el gradiente de la función objetivo. Dependiendo de la cantidad de datos, tenemos una trade-off entre la precisión del update de los pesos y el tiempo para llevar a cabo ese update.

Tipos de gradient descents

1. Batch GD:

Aquí se procesan todas las instancias de entrenamiento para cada iteración del GD. Si el número de instancias es demasiado grande, entonces esto es computacionalmente caro. Si este es el caso, el batch GD no es la mejor alternativa. Mejor usar stochastic GD o mini batch GD.

Tipos de gradient descents

2. Stochastic GD:

Aquí se procesa 1 instancia de entrenamiento por iteración. Así, los pesos (gradientes) se actualizan por cada instancia que entra (de a una). Esto es más rápido que el batch GD, pero nuevamente, si el número de instancias es muy grande, hacer el proceso de actualización instancia por instancia podría ser un poco lento.

Tipos de gradient descents

3. Mini Batch GD:

Por lo general este es el que usamos. Es más rápido que los otros dos, En este caso una cantidad $b < m$ se procesan por iteración (donde m es el número de instancias de entrenamiento disponible).

Así, si tenemos muchas instancias, el procesamiento se hace en “lotes” (batches). Por eso es más adecuado cuando tenemos muchas instancias de entrenamiento.

Aclaraciones - Batch size

En el proceso de fitting de la NN a los datos de entrenamiento nos encontramos con algo como esto:

```
model.fit(x_train_data, y_train_targets, epoch=XYZ, batch_size=XYZ)
```

Aclaraciones - Batch size

En el proceso de fitting de la NN a los datos de entrenamiento nos encontramos con algo como esto:

```
model.fit(x_train_data, y_train_targets, epoch=XYZ, batch_size=XYZ)
```

El batch es un hiperparámetro que define el número de muestras a trabajar (del train set) que se van a propagar en el proceso de feedforward/backforward.

batch_size = number of samples per gradient update (Keras)

Supongamos que tenemos 1050 muestras de entrenamiento y batch_size=100. El algoritmo toma primeras 100 muestras del set de training y entrena la red con esas 100. Luego toma las siguientes 100 y así sucesivamente. Se hace esto hasta que todas las muestras se hayan propagado a través de la red.

Aclaraciones - Batch size

En el proceso de fitting de la NN a los datos de entrenamiento nos encontramos con algo como esto:

```
model.fit(x_train_data, y_train_targets, epoch=XYZ, batch_size=XYZ)
```

El batch es un hiperparámetro que define el número de muestras a trabajar (del train set) que se van a propagar en el proceso de feedforward/backforward.

Ventajas:

Se requiere menos memoria. Al entrenar la red con menos muestras, el procedimiento requiere menos memoria, lo cual es importante cuando no es posible hacer fit a todo el dataset en la memoria del computador.

Además el entrenamiento es más rápido con minibatches. Esto es porque el número de updating de los pesos en cada backpropagation es menor. En el ejemplo, propagamos 11 batches (10 de ellos con 100 muestras) y después se actualizan los parámetros de la red. Si

Aclaraciones - Batch size

En el proceso de fitting de la NN a los datos de entrenamiento nos encontramos con algo como esto:

```
model.fit(x_train_data, y_train_targets, epoch=XYZ, batch_size=XYZ)
```

El batch es un hiperparámetro que define el número de muestras a trabajar (del train set) que se van a propagar en el proceso de feedforward/backforward.

Desventajas:

Mientras más pequeño el tamaño del batch, la estimación de los gradientes es menos precisa!

Aclaraciones - Batch size

En el proceso de fitting de la NN a los datos de entrenamiento nos encontramos con algo como esto:

```
model.fit(x_train_data, y_train_targets, epoch=XYZ, batch_size=XYZ)
```

El batch es un hiperparámetro que define el número de muestras a trabajar (del train set) que se van a propagar en el proceso de feedforward/backforward.

Tipos:

(Vanilla) Gradient descent:

Batch size = size of training set

Stochastic Gradient descent:

Batch size = 1

Mini-batch Gradient descent:

$1 < \text{batch size} < \text{size of training set}.$

Aclaraciones - Batch size

En el proceso de fitting de la NN a los datos de entrenamiento nos encontramos con algo como esto:

```
model.fit(x_train_data, y_train_targets, epoch=XYZ, batch_size=XYZ)
```

El batch es un hiperparámetro que define el número de muestras a trabajar (del train set) que se van a propagar en el proceso de feedforward/backforward.

Tipos:

(Vanilla) Gradient descent:

Batch size = size of training set

El gradiente se calcula sobre todos los data points en forma individual, y luego se calcula un promedio. Luego, se hace el (único) update a los pesos.

Aclaraciones - Batch size

En el proceso de fitting de la NN a los datos de entrenamiento nos encontramos con algo como esto:

```
model.fit(x_train_data, y_train_targets, epoch=XYZ, batch_size=XYZ)
```

El batch es un hiperparámetro que define el número de muestras a trabajar (del train set) que se van a propagar en el proceso de feedforward/backforward.

Tipos:

Stochastic Gradient descent:

Batch size = 1

Aquí se calcula el gradiente en cada data point y hay tantas actualizaciones como data points hayan. Este método es más “oscilatorio” lo cual puede hacer que el algoritmo no converga (o le cueste llegar al mínimo).

Aclaraciones - Batch size

En el proceso de fitting de la NN a los datos de entrenamiento nos encontramos con algo como esto:

```
model.fit(x_train_data, y_train_targets, epoch=XYZ, batch_size=XYZ)
```

El batch es un hiperparámetro que define el número de muestras a trabajar (del train set) que se van a propagar en el proceso de feedforward/backforward.

Tipos:

Mini-batch Gradient descent: $1 < \text{batch size} < \text{size of training set}.$

Esta es una combinación de las dos anteriores porque se toman una cantidad de “lotes” de data points. Por cada uno de estos se calculan los gradientes y se obtiene un promedio.

Aclaraciones - Epoch

El epoch también es un hiperparámetro que define el número de veces que el algoritmo de aprendizaje va a ejecutarse a través de **TODO el set de datos de entrenamiento**. Es decir, el número de forward y backward propagations de todas las muestras del dataset.

Por ejemplo, si el data set de entrenamiento tenemos 1000 ejemplos y el tamaño del batch es 500, entonces el proceso tomará 2 iteraciones completar 1 epoch.